CPU Virtualization
CPU Time share scheduling

Sk Subidh Ali
IIT Bhilai

Interactive process scheduling

Aften time we use interactive programs which

- Takes input from keyboard/mouse/touch screen
- Produce output at VDU/monitor

Getting quick response is crucial in interactive system

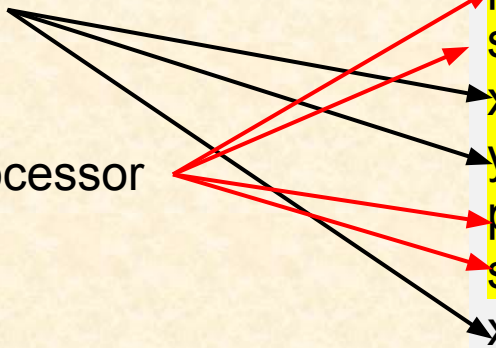
Response time: (Time of first run – Time of arrival**)**

CPU bound: Executed in CPU

Short CPU burst time

I/O bound: Executed in I/O processor

Keep waiting for I/O



```
#include<stdio.h>

int main()
{
int x, y,z;
char name[30], dmpt[10],
addrs[50];

printf("\n Enter a value:");
scanf("%d",name);
x=x*x;
y=x+5
printf("\nEnter a value");
scanf("%d",z);
x=y/5+z
…
…
return(0);
}
```

Response time

Response time: Time of first run – Time of arrival

Find average **Response Time** in SRTF ???

$$\frac{(0+12)+(0)+(6-2)}{3} = 5.33$$

For SJF or SRTF, response time is same as the average waiting time.

- How can we democratize CPU
- Reduce response and turn around time together

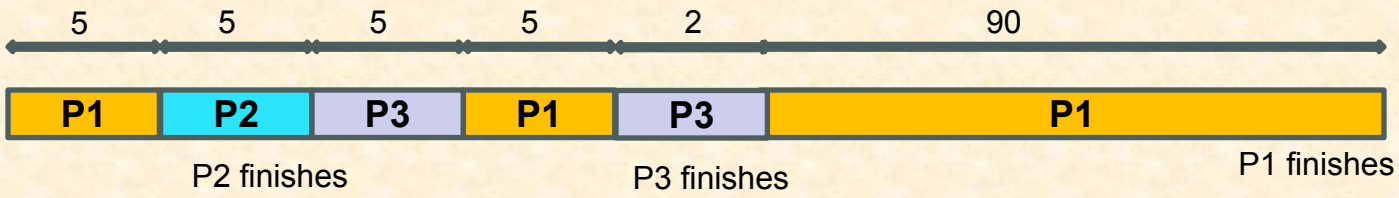
Round Robin Scheduling

- Scheduler executes each process for a fix amount of CPU time called time slice or ***quanta***
- Once a process finishes a ***quanta*** execution:
  - timer interrupts and scheduler is invoked
  - scheduler runs the next process in ready queue

CPU virtualization: Each running process feels that it has it's one CPU

Assume Quanta=5 unit

Process	Arrival time	CPU burst time
P1	0	100
P2	1	5
P3	2	7



Round Robin Scheduling

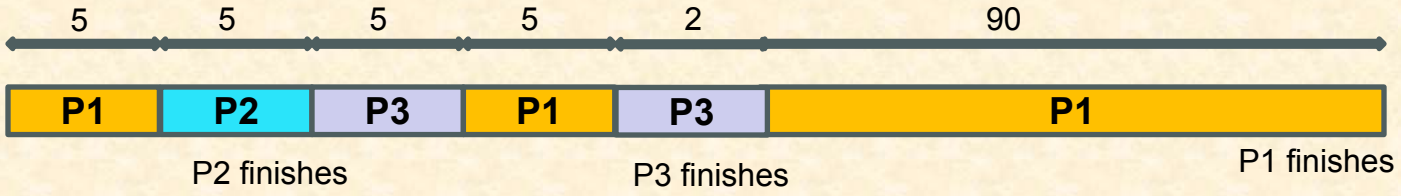
Average waiting time: $\frac{(5+5+5)+(5-1)+(10-2+5)}{3} = 10.66$

Turn around time: $\frac{112+(10-1)+(25-2)}{3} = 48$

Response time: $\frac{0+(5-1)+(10-2)}{3} = 4$

Is it good?

Process	Arrival time	CPU burst time
P1	0	100
P2	1	5
P3	2	7



RR vs SRTF

RR:

Average waiting time: $\frac{(5+5+5)+(5-1)+(10-2+5)}{3}=10.66$

Turn around time: $\frac{112+10-1+25-2}{3}=48$

Response time: $\frac{0+(5-1)+(10-2)}{3}=4$

SRTF:

Average waiting time: $\frac{(0+12)+(0)+(6-2)}{3}=5.33$

Turn around time: $\frac{112+(6-1)+(13-2)}{3}=42.6$

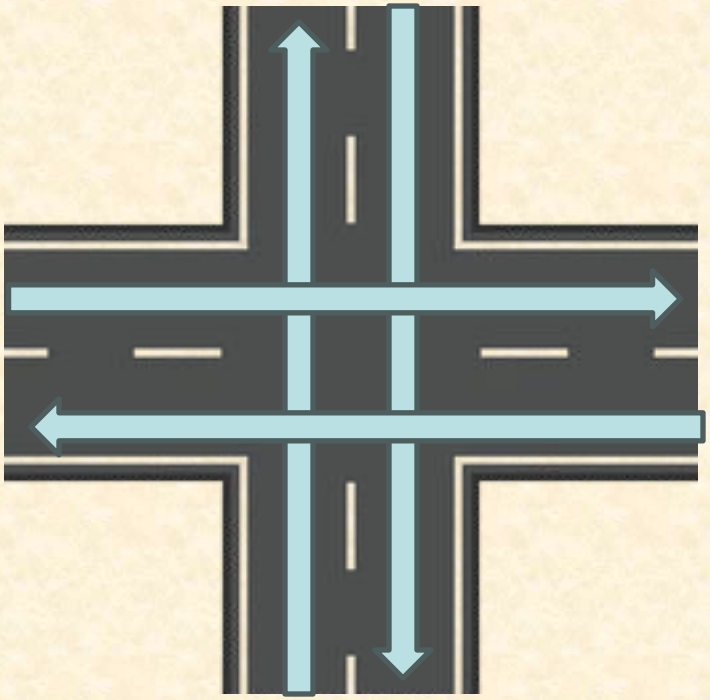
Response time: $\frac{(0+12)+(0)+(6-2)}{3}=5.33$

OS can not predict how long a process will run

	RR	SRTF
Fair/democratic	Yes	No
Responsive	Better	good
Overhead	More context switches	Less context switches
Practical	Yes	No

RR in Traffic Management

- There are four mutual exclusive traffics shown in arrow
- Each gets equal amount time to pass
- Example of RR scheduling



Priorities matters

In real world, priorities are different for different entities in the system.

Which one has high priority?

What about priority in CPU scheduling?

Which one has high priority?

RR with priority scheduling

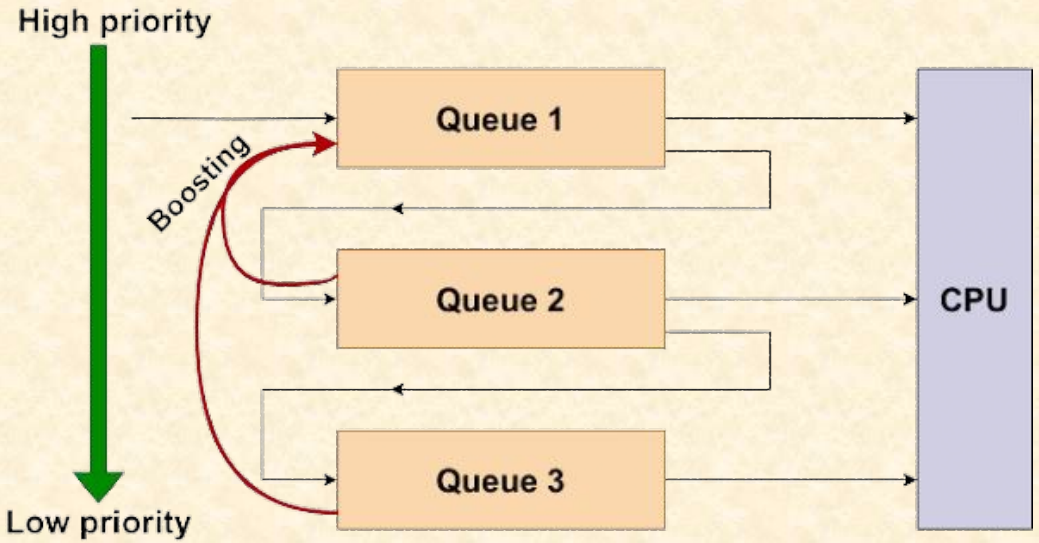
- **Priority scheduling** needs to *know in advance* how long a process will run or how important it is.
- OS can not guess length of execution or priority for a general application program
  - Process execution length changes run time based on conditions defined in if else { }

How do we do it then?

- OS must adjust priority during process run time
- Assign high priority to new processes. Then adjust priorities based on observed behaviors
- High priority will get small CPU chunk (**quanta value**)

Multi-Level Feedback Queue(MLFQ)

Fernando J. Corbató
PhD MIT

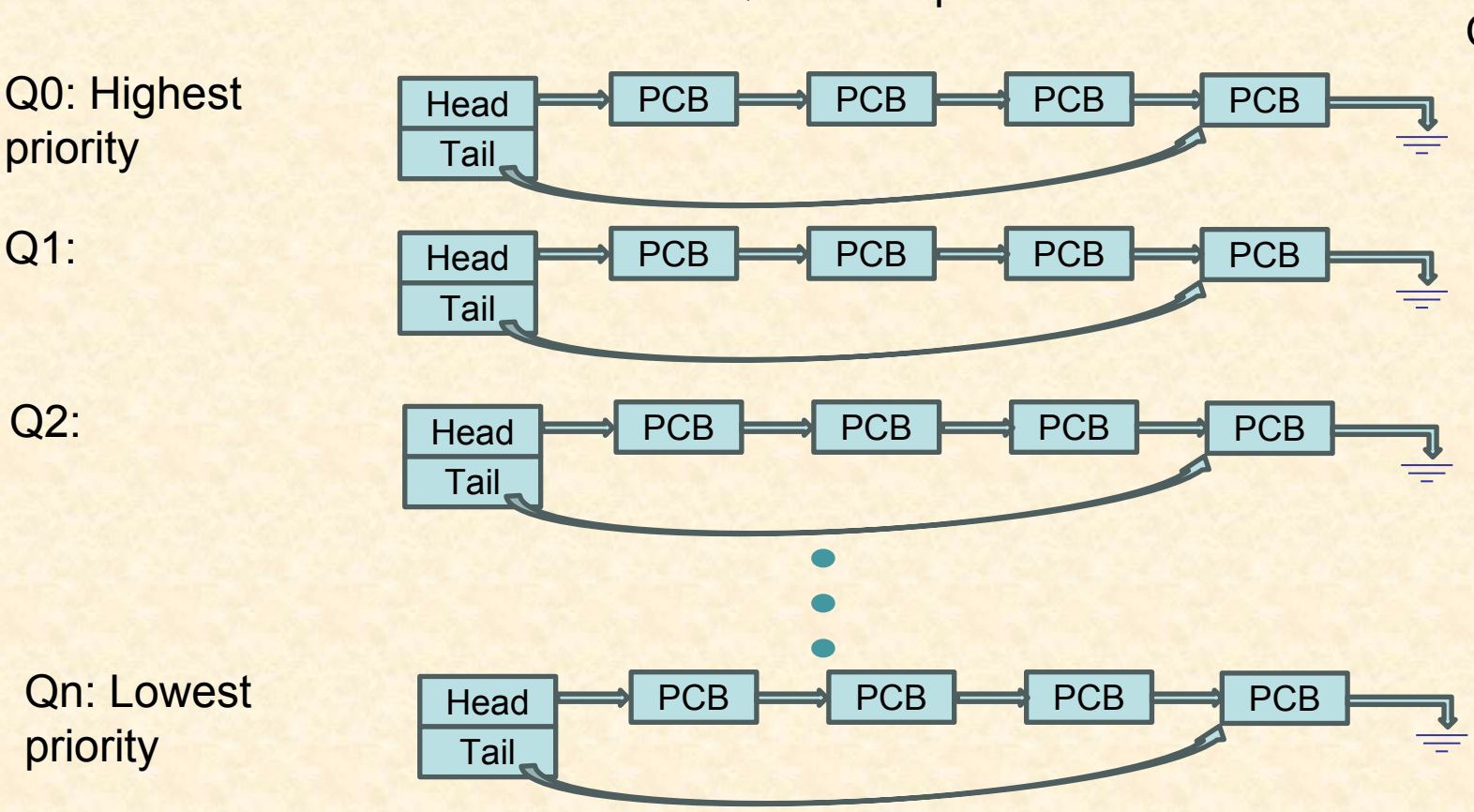


1990 Turing award for his work on Compatible time-sharing system (CTSS) in 1961
CTSS led the development of first secure OS Multics
Multics led to development of Unix and later Linux

Multi-Level Feedback Queue(MLFQ)

MLFQ performs priority scheduling by maintaining multiple queues

- Modern windows OS uses MLFQ with 32 queues



Quanta value increase as priority decreases

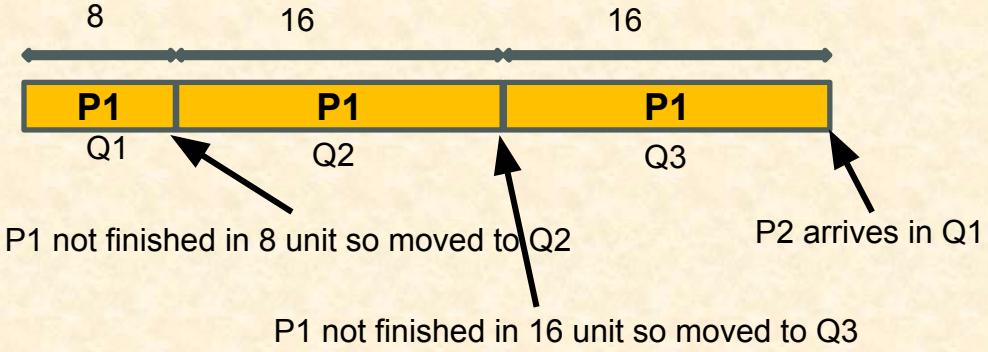
C1<C2<C3 … <Cn

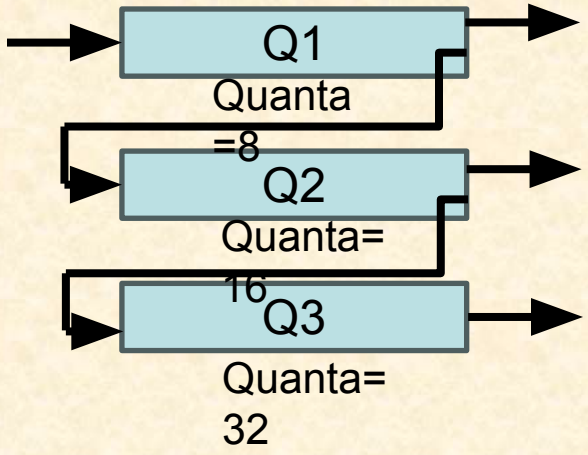
Multi-Level Feedback Queue(MLFQ)

Key principle:

1. A new job enters at the highest priority queue
   - it might be short/interactive and might finish quickly
2. If a job uses its entire time slice without giving up the CPU,
   - its priority is reduced — it drops down one queue.
3. If a job gives up the CPU before its time slice expires
   - (e.g., it does I/O and blocks),
   - it **stays at the same priority level**.

Process	Arrival time	CPU burst time
P1	0	100
P2	40	12





Multi-Level Feedback Queue(MLFQ)

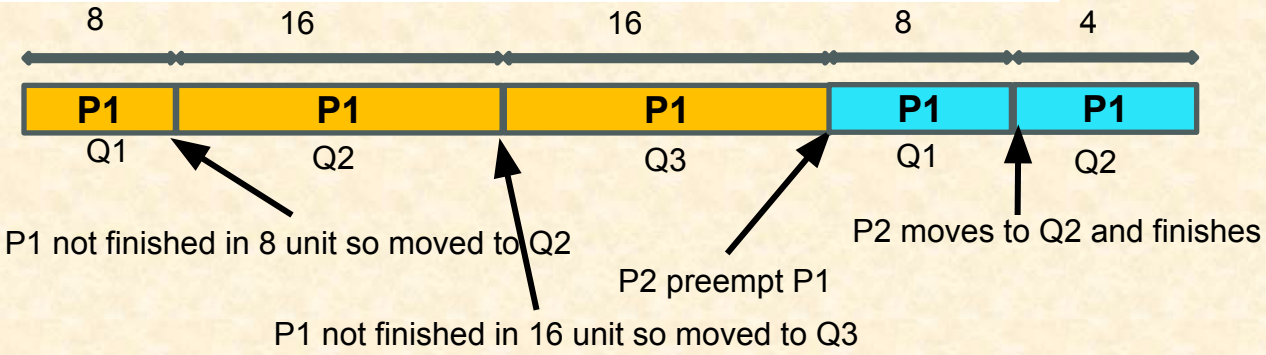
Key principle:

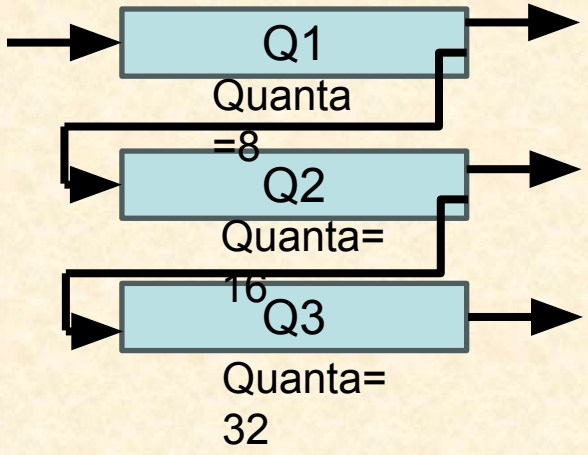
4. If Priority(A) > Priority(B),
   - A runs (B doesn't).
5. If Priority(A) == Priority(B),
   - A and B run in round robin using that queue's time slice.

Priority (P2) >Priority(P1) as P2 in Q1 and P1 in Q3

After P2, P1 execute in Q3 in RR with quanta=32

Process	Arrival time	CPU burst time
P1	0	100
P2	40	12

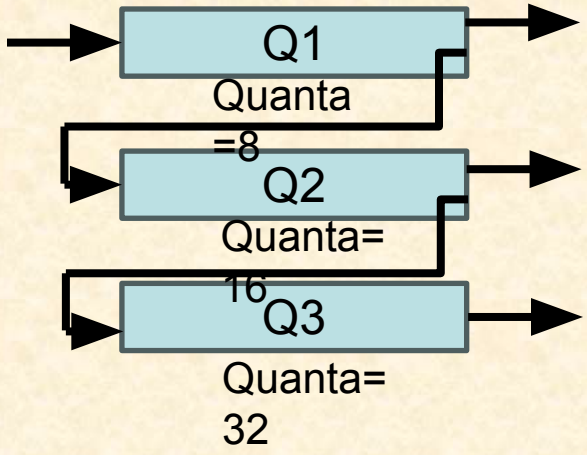




Multi-Level Feedback Queue(MLFQ)

Key principle:

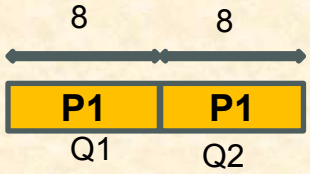
- If Priority(A) > Priority(B),
  - A runs (B doesn't).
- If Priority(A) == Priority(B),
  - A and B run in round robin using that queue's time slice.
- A new job enters at the highest priority queue
  - it might be short/interactive and might finish quickly
- If a job uses its entire time slice without giving up the CPU,
  - its priority is reduced — it drops down one queue.
- If a job gives up the CPU before its time slice expires
  - (e.g., it does I/O and blocks),
  - it stays at the same priority level.

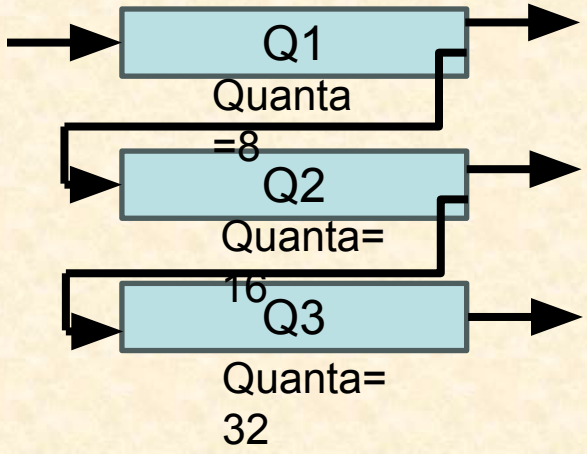


Multi-Level Feedback Queue(MLFQ)

What if P3 comes at 16?

Process	Arrival time	CPU burst time
P1	0	100
P2	40	12
P3	16	6



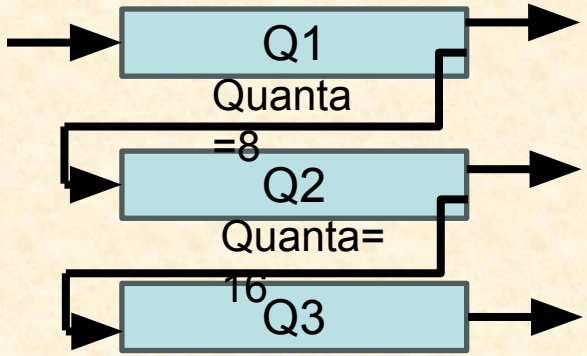


Challenge in MLFQ

1. **Starvation.** If there are enough interactive/high-priority process, low-priority (CPU-bound) process might never get the CPU at all.

Example:

Process	Arrival time	CPU burst time
P1	0	100
P2	10	7
P3	16	6
P4	20	7
P5	25	8



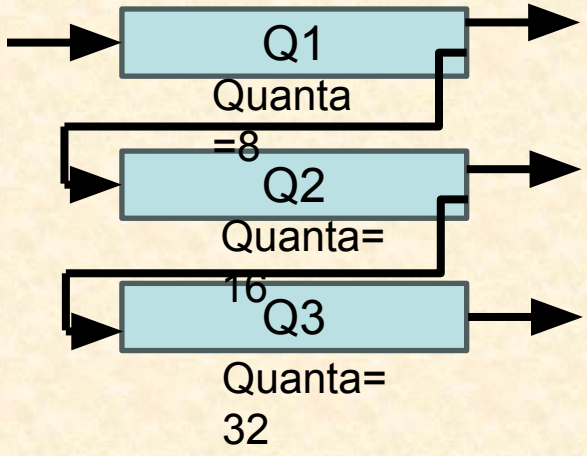
P1 will first execute in Q1 for 8 unit and goes to Q2.
It will not get a chance to execute before P2,P3,P4,P5 finishes

Challenge in MLFQ

2. Scheduler can be fooled. One can deliberately add I/O operation just before its time slice expires and keep continue in high priority

*Say in RISC-V **n** instructions are equivalent to 7 quanta time. I will put an **out** instruction after every **n** instruction. Then my process will always execute in Q1, as it will never complete quanta (8 unit)*

You can launch DOS (denial of service) attack

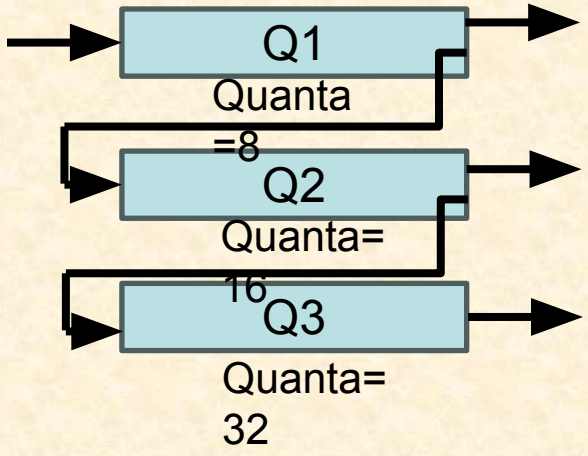


Priority boosting in MLFQ

Rule 5: After some time period S, move all jobs in the system to topmost queue (highest priority)

It avoids Starvation: As each low priority process get another chance to execute as high priority process

Avoid getting fooled by better accounting:
Once a process uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).



Next class IPC: Galvin Ch3.4